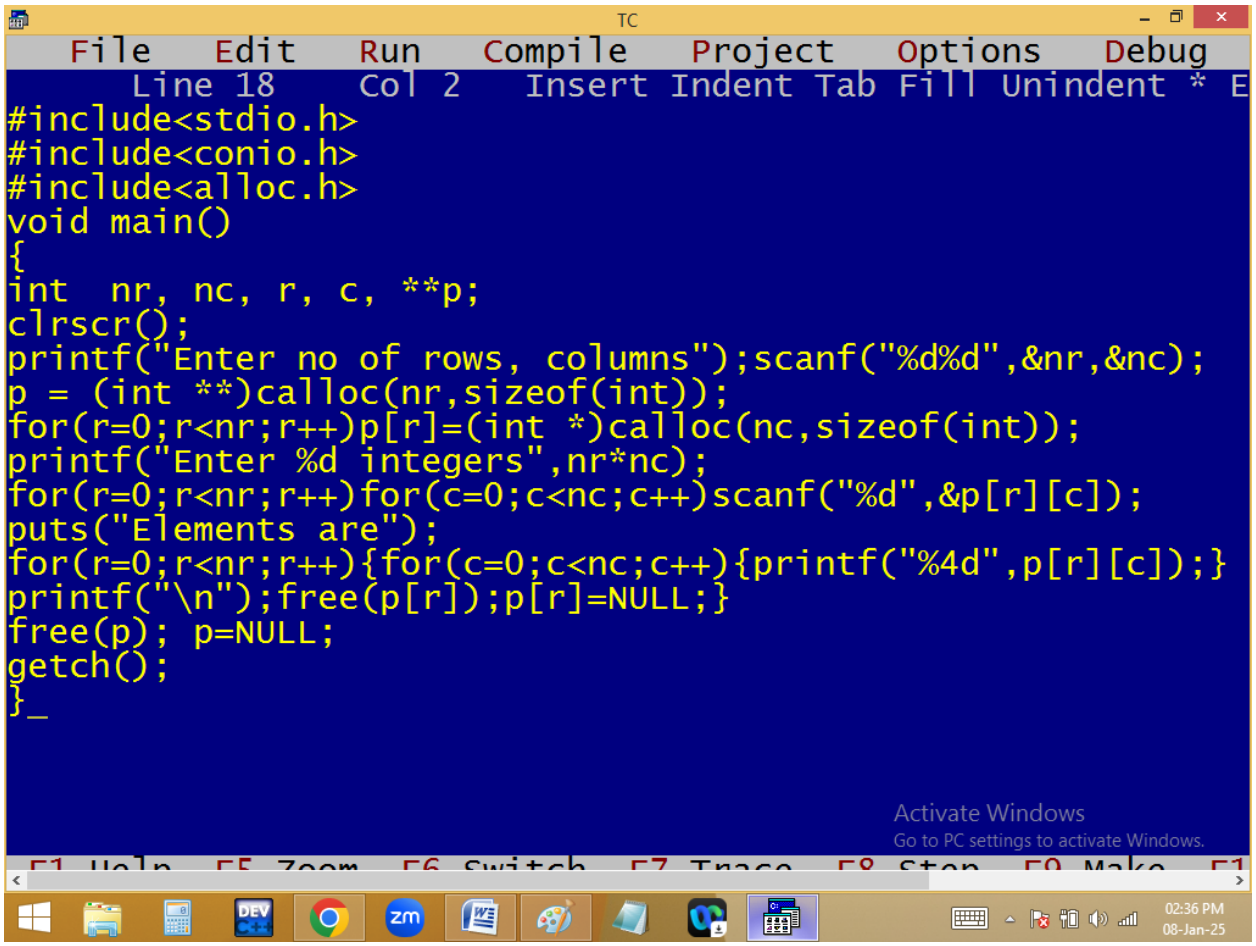
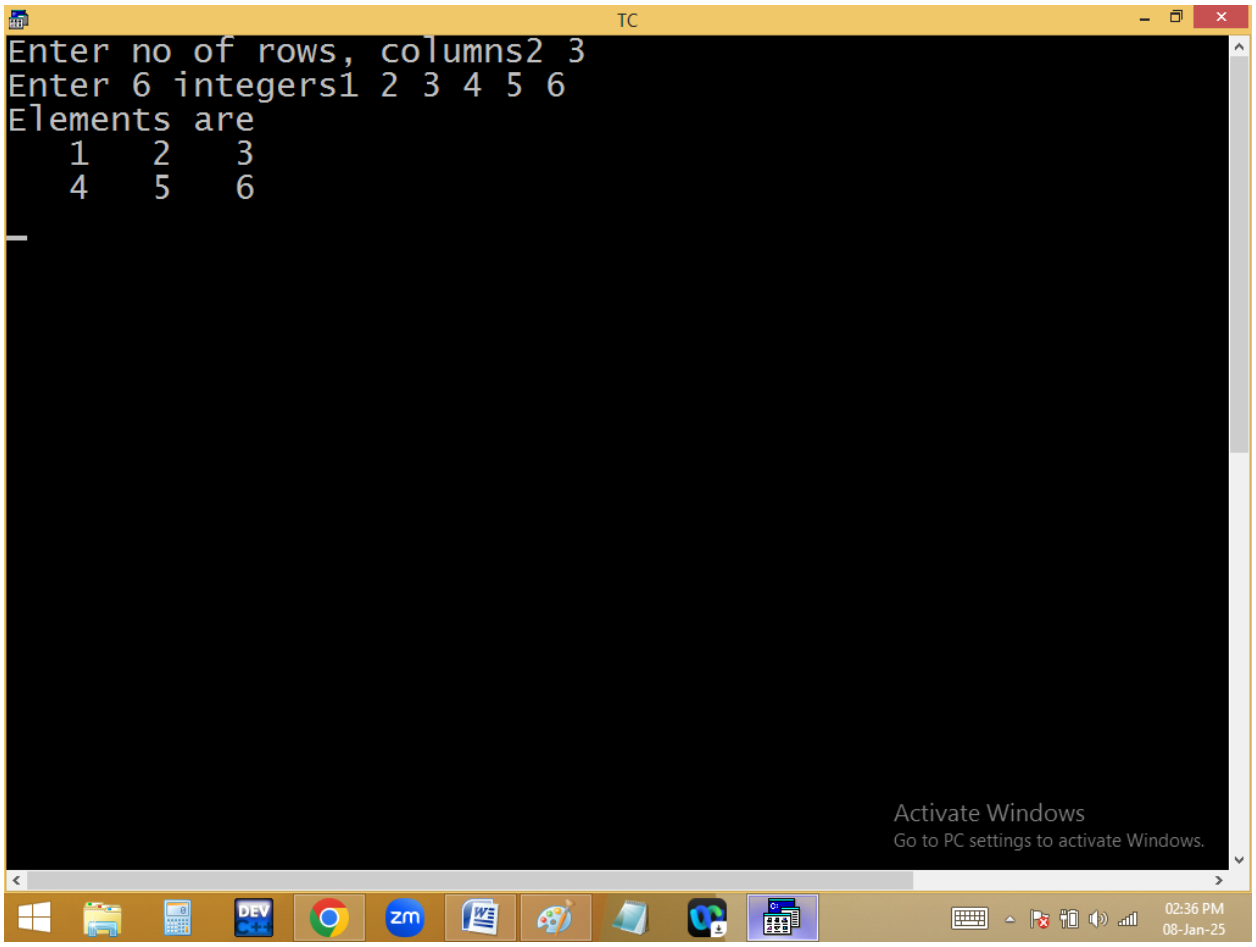
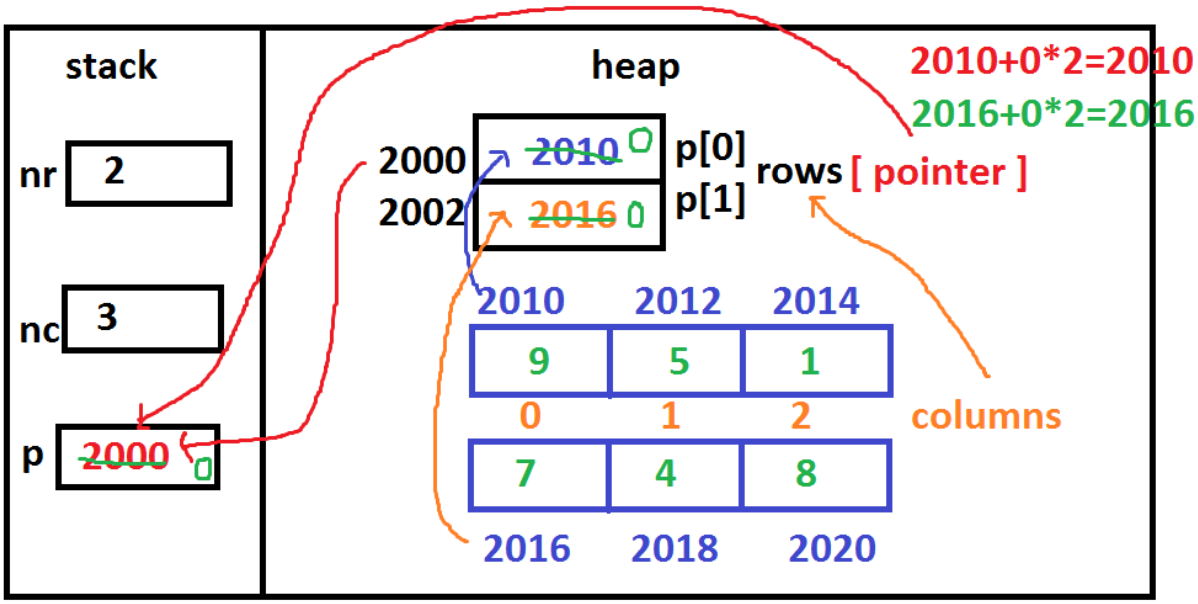
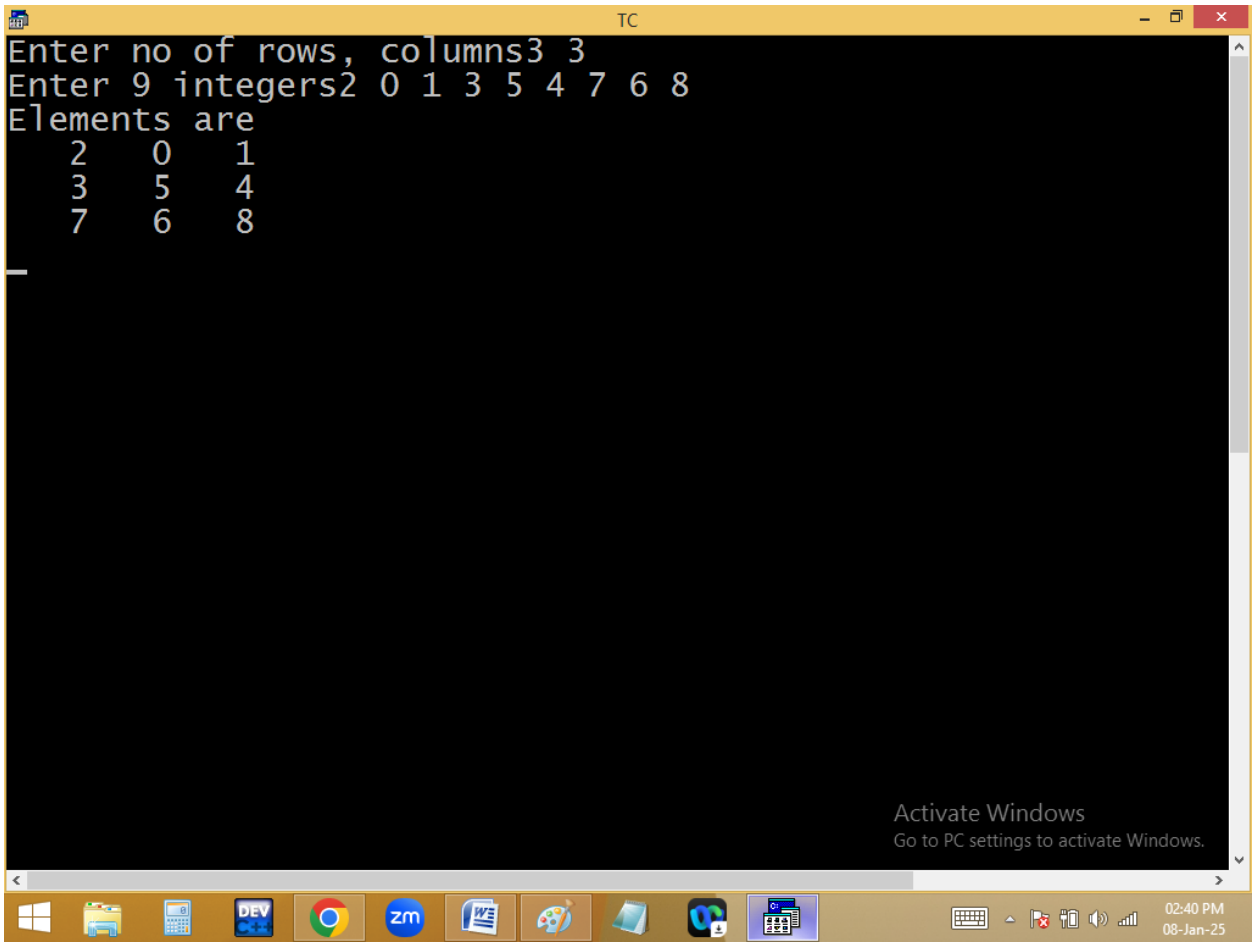
Creating a dynamic multi dimensional array:


TC
File Edit Run Compile Project Options Debug
Line 18 Col 2 Insert Indent Tab Fill Unindent * E
#include<stdio.h>
#include<conio.h>
#include<alloc.h>
void main()
{
int nr, nc, r, c, **p;
clrscr();
printf("Enter no of rows, columns");scanf("%d%d",&nr,&nc);
p = (int **)calloc(nr,sizeof(int));
for(r=0;r<nr;r++)p[r]=(int *)calloc(nc,sizeof(int));
printf("Enter %d integers",nr*nc);
for(r=0;r<nr;r++)for(c=0;c<nc;c++)scanf("%d",&p[r][c]);
puts("Elements are");
for(r=0;r<nr;r++){for(c=0;c<nc;c++){printf("%4d",p[r][c]);}
printf("\n");free(p[r]);p[r]=NULL;}
free(p); p=NULL;
getch();
}
Activate Windows
Go to PC settings to activate Windows.
02:36 PM
08-Jan-25


TC
Enter no of rows, columns2 3
Enter 6 integers1 2 3 4 5 6
Elements are
1 2 3
4 5 6
Activate Windows
Go to PC settings to activate Windows.
02:36 PM
08-Jan-25

```
TC
File  Edit  Run  Compile  Project  Options  Debug
     Line 15    Col 19  Insert Indent Tab Fill Unindent * E
#include<stdio.h>
#include<conio.h>
#include<alloc.h>
void main()
{
int  nr, nc, r, c, **p;
clrscr();
printf("Enter no of rows, columns");scanf("%d%d",&nr,&nc);
p = (int **)calloc(nr,sizeof(int));
for(r=0;r<nr;r++)p[r]=(int *)calloc(nc,sizeof(int));
printf("Enter %d integers",nr*nc);
for(r=0;r<nr;r++)for(c=0;c<nc;c++)scanf("%d",*(p+r)+c);
puts("Elements are");
for(r=0;r<nr;r++){for(c=0;c<nc;c++)
{printf("%4d",*(*(p+r)+c));}
printf("\n");free(p[r]);p[r]=NULL;}
free(p); p=NULL;
getch();
}
```





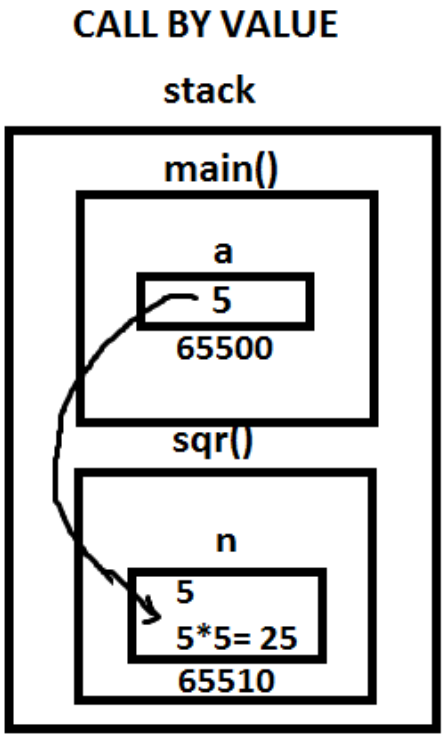
Passing parameters to the functions: [parameter passing techniques]

In C-Language, we can send the arguments to the functions in 2 ways.

1. **Call by value / pass by value.**
2. **Call by address / pass by address. [call by reference]**

Call by value / pass by value:

In call by value we are sending actual parameter value to the formal parameter. Later there is no relation is maintained in between actual and formal parameters. Due to this any change in formal parameter doesn't effects the value of actual parameter.



Eg: 1

```
#include<stdio.h>
#include<conio.h>

void  sqr(int n)

{

n = n * n;

}  /* n  deleted after the function execution
*/

void main()

{

int a=5;

clrscr();

printf("Before function call a = %d\n" ,a);

sqr(a);  /* fun calling */

printf("After function call a = %d", a):

getch();

}
```

Output:

Before function call a = 5

After function call a = 5

Eg: 2 swapping of two integers

```
#include<stdio.h>
#include<conio.h>
void swap(int a, int b)
{
int temp=a;
a=b;
b=temp;
}
void main()
{
int a=5, b=7;
clrscr();
printf("Before fun call a=%d, b=%d\n" , a , b);
swap(a, b);
printf("After fun call a=%d, b=%d", a , b);
```

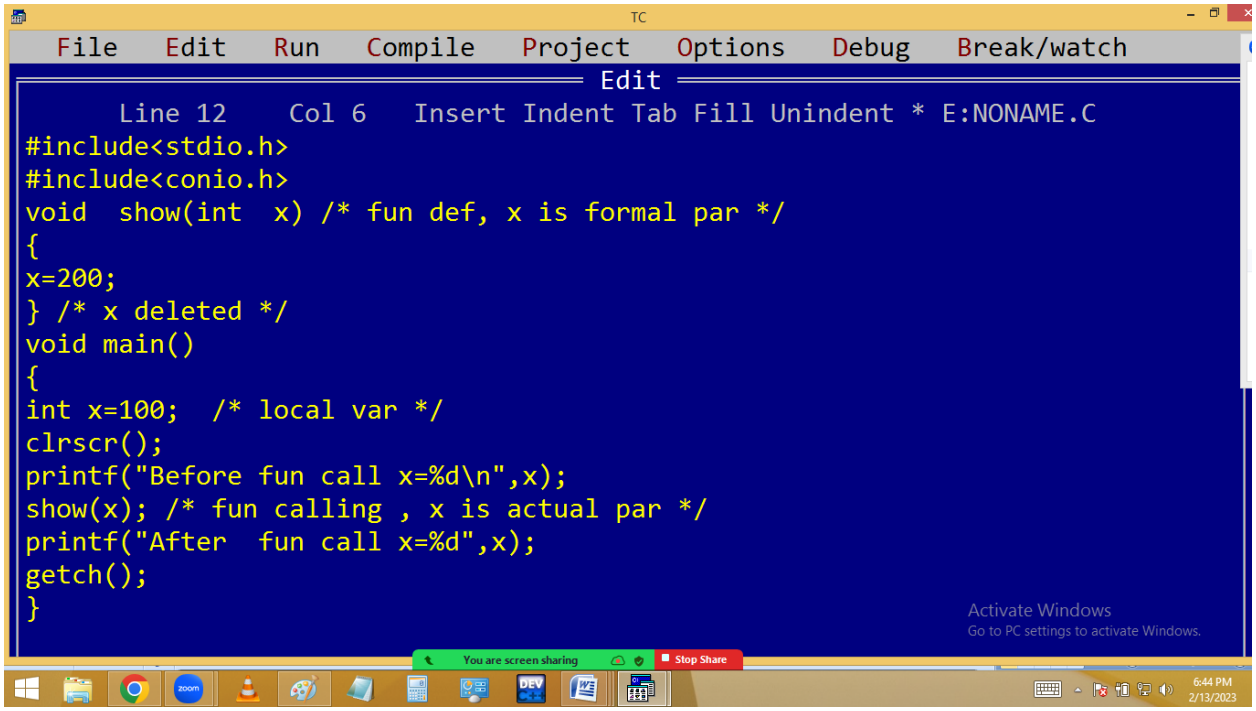
getch();

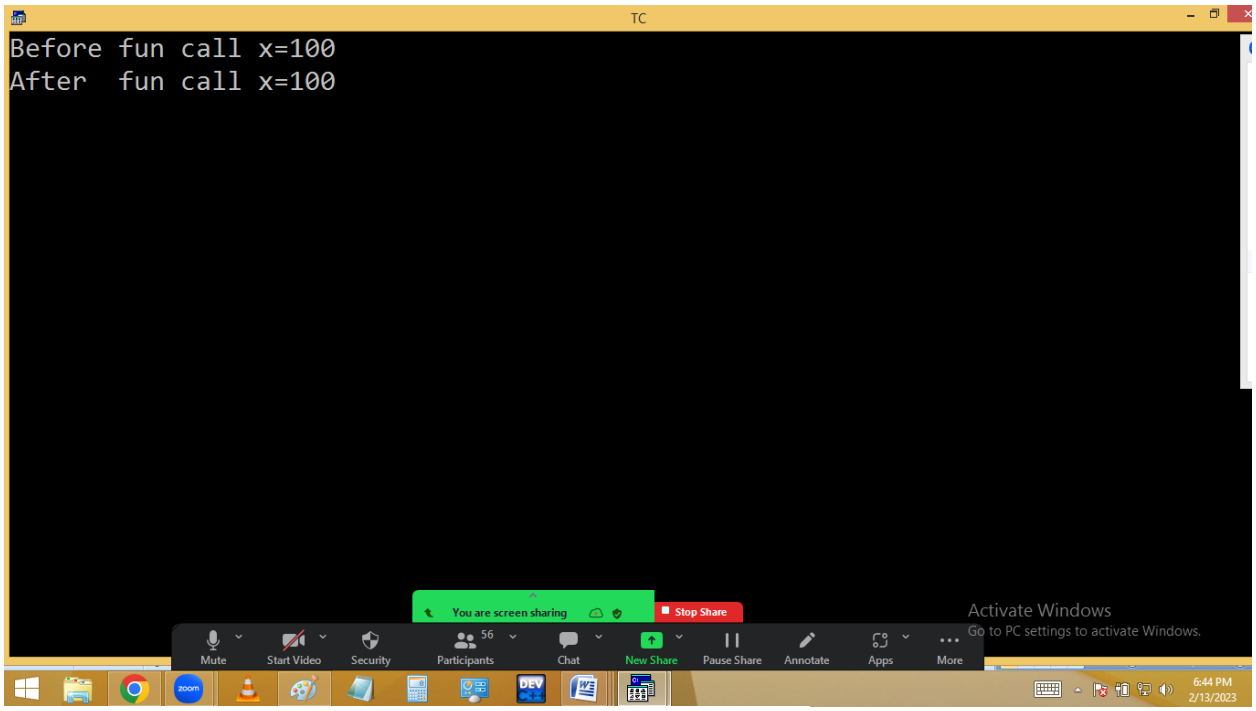
}

Output:

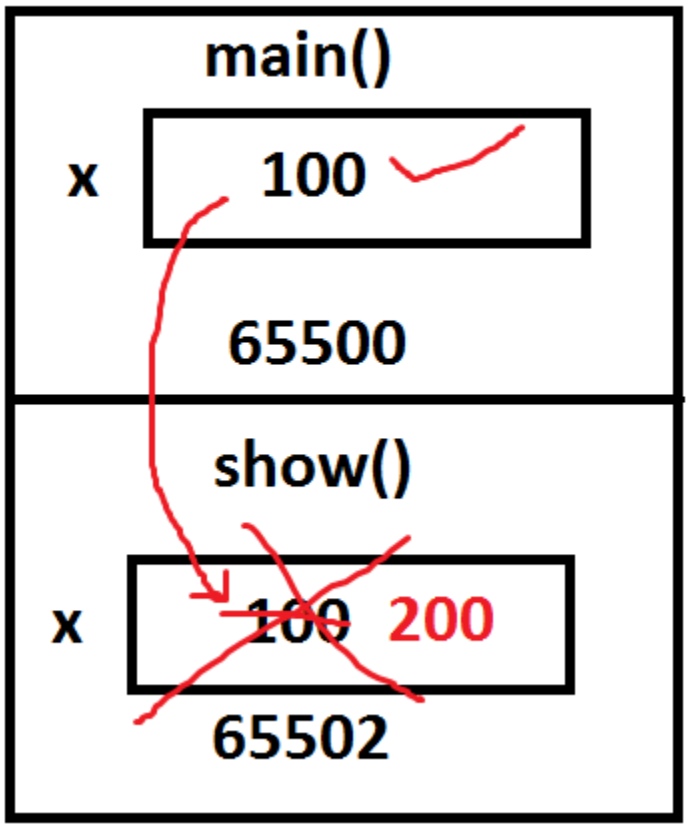
Before fun call a=5, b=7

After fun call a=5, b=7





Call by /pass By value



Call by address [Reference]:

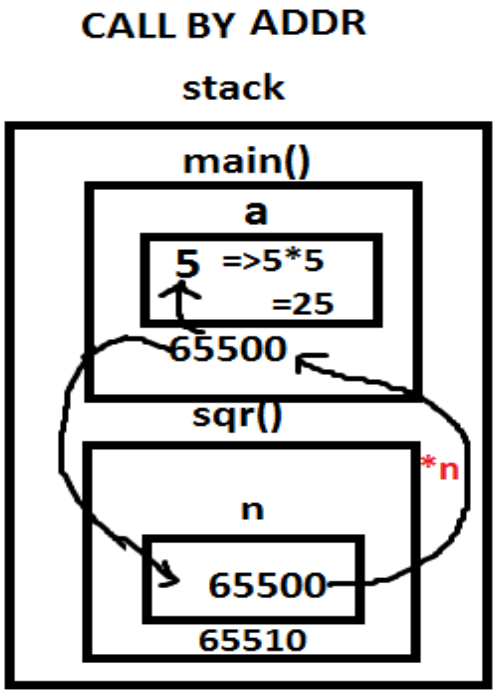
In call by address, the address of actual parameter is passed to formal parameter. Due to this the formal parameter should be declared as a pointer. Then only the formal parameter receives the actual parameter address. Due to this any changes in formal parameter effects in actual parameter address i.e. actual parameter value.

Hence pointers allows the local variables to access outside the functions and this process is called call by address / reference.

It is very much useful in handling the strings, arrays etc outside the functions.

Eg: 1

```
#include<stdio.h>
#include<conio.h>

void  sqr(int *n)

{

*n = *n   *   *n;

}

void main()

{

int a=5;

clrscr();

printf("Before function call a = %d\n " ,
```



```
a);
sqr(&a);  /* fun calling with address */
printf("After function call a = %d " , a);
getch();
}
```

Output:

Before function call a = 5

After function call a = 25

Eg: 2 Swap of two integers

```
#include<stdio.h>
#include<conio.h>
void  swap(int  *a, int  *b)
{
int  temp=*a;   *a = *b;  *b=temp;
}
void  main()
{
int  a=5, b=7;
clrscr();
```

printf("Before fun call a=%d, b=%d\n", a ,b);

swap(&a, &b);

printf("After fun call a=%d, b=%d", a ,b);

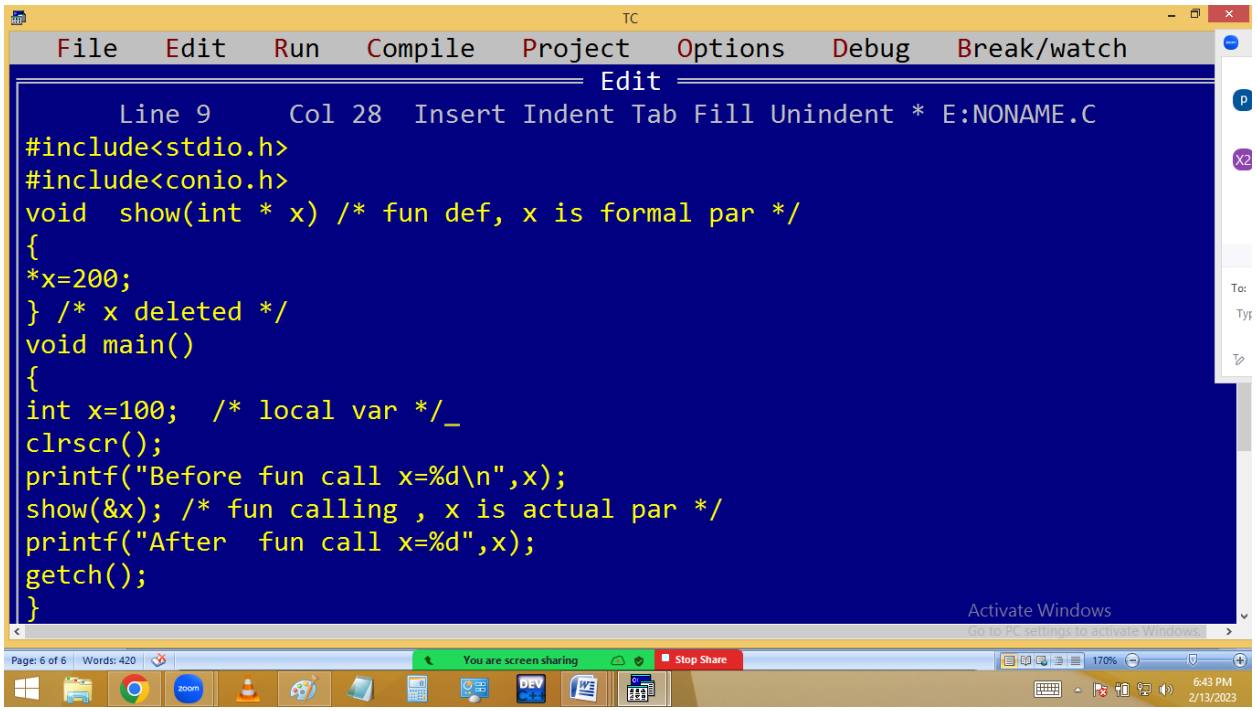
getch();

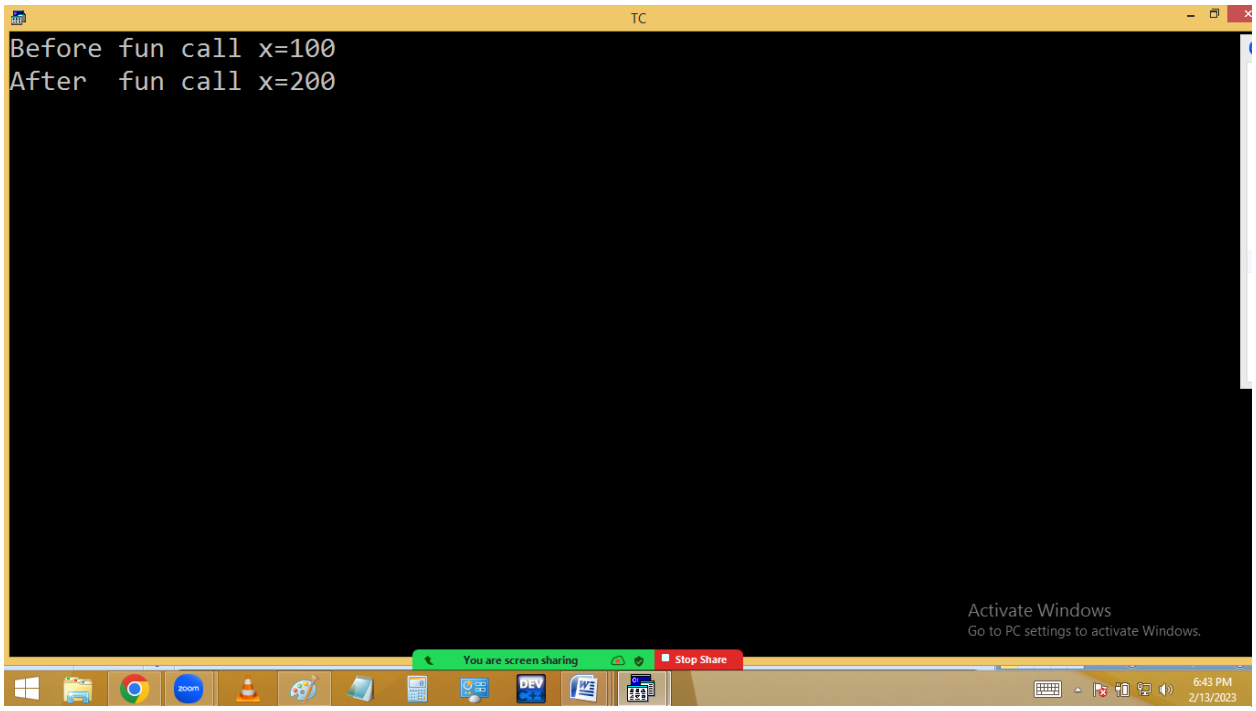
}

Output:

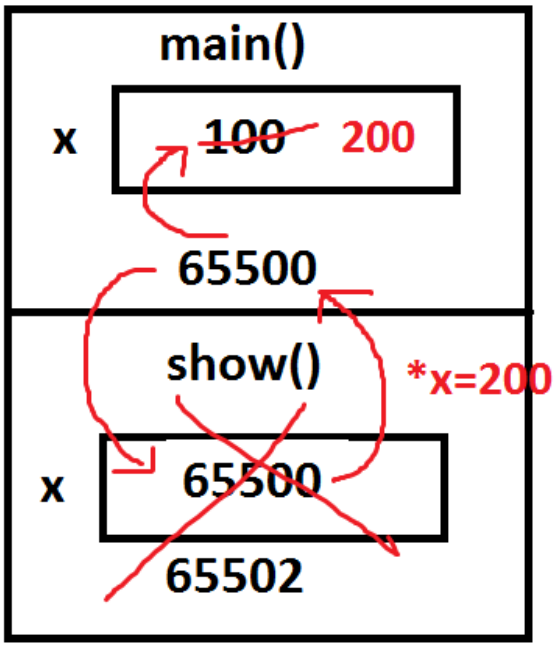
Before function call a=5, b=7

After function call a=7, b=5





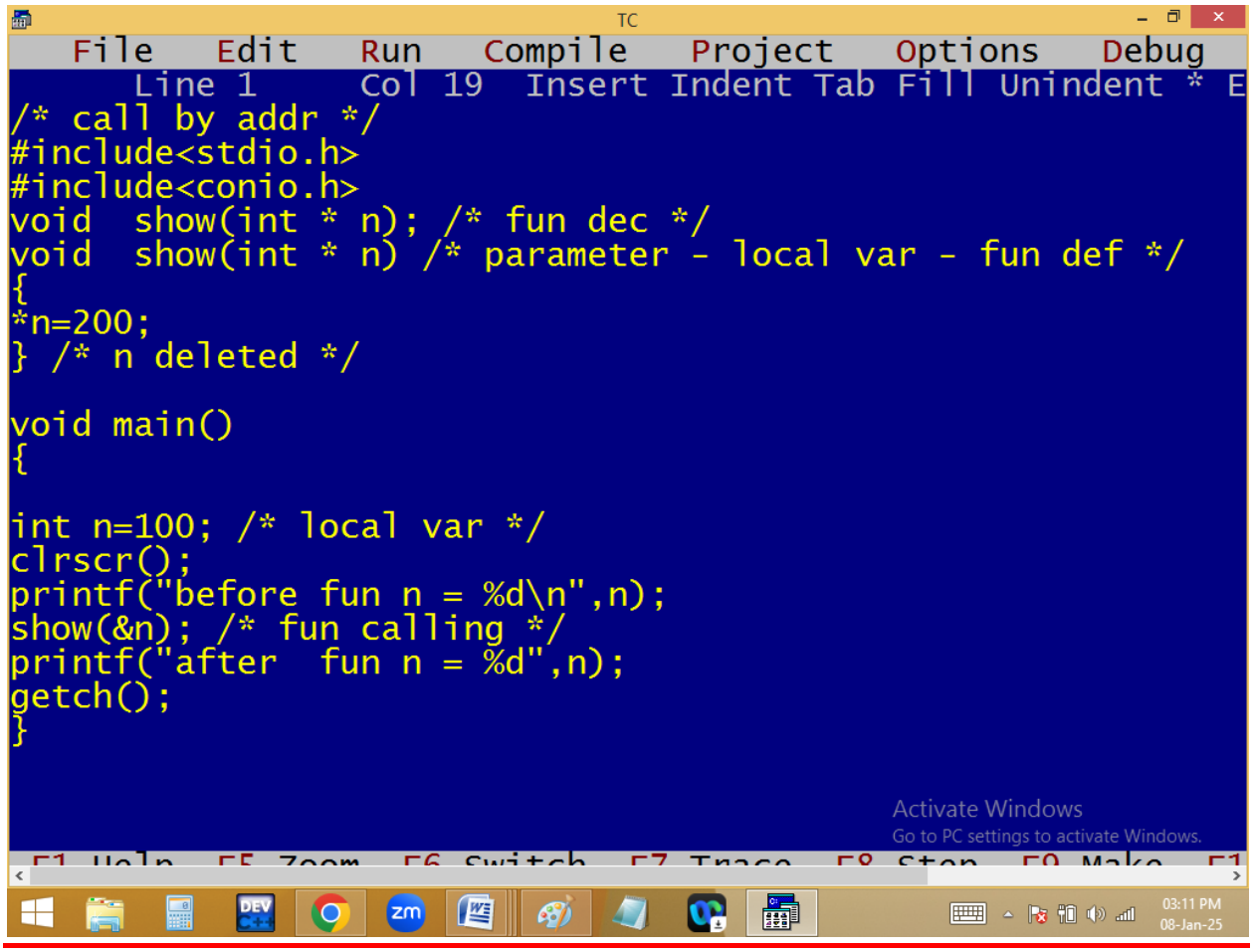
call / pass by address

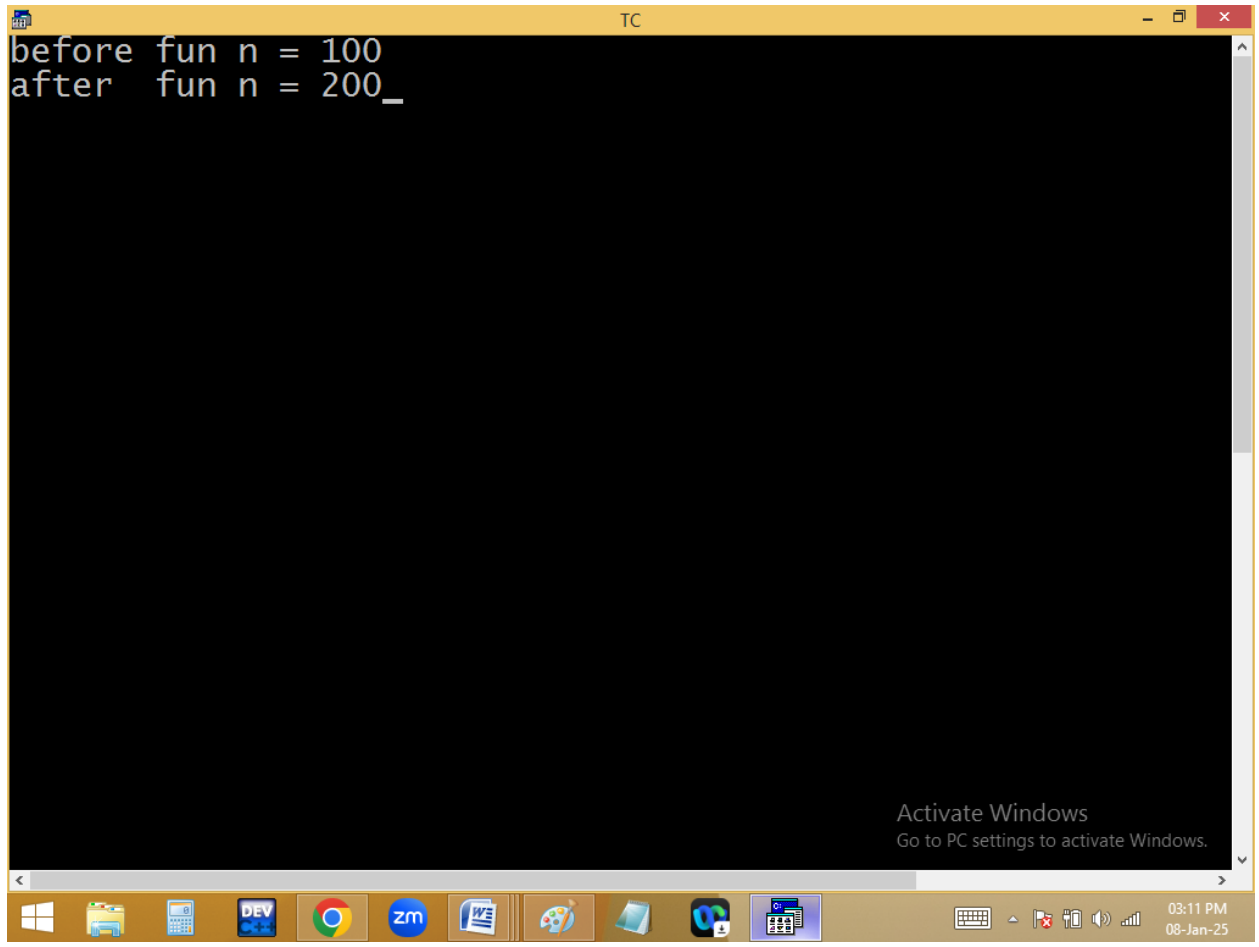


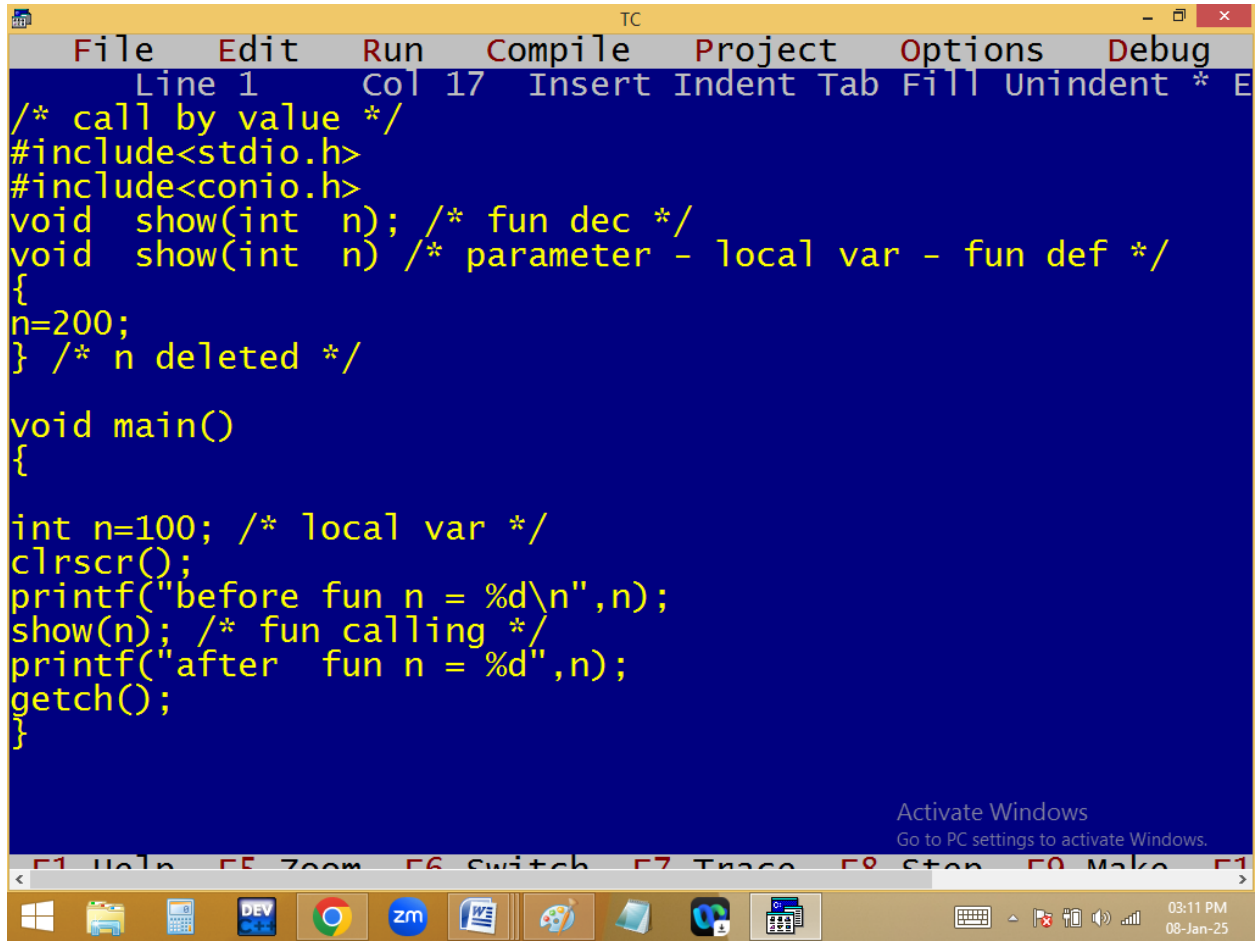
*x = 200

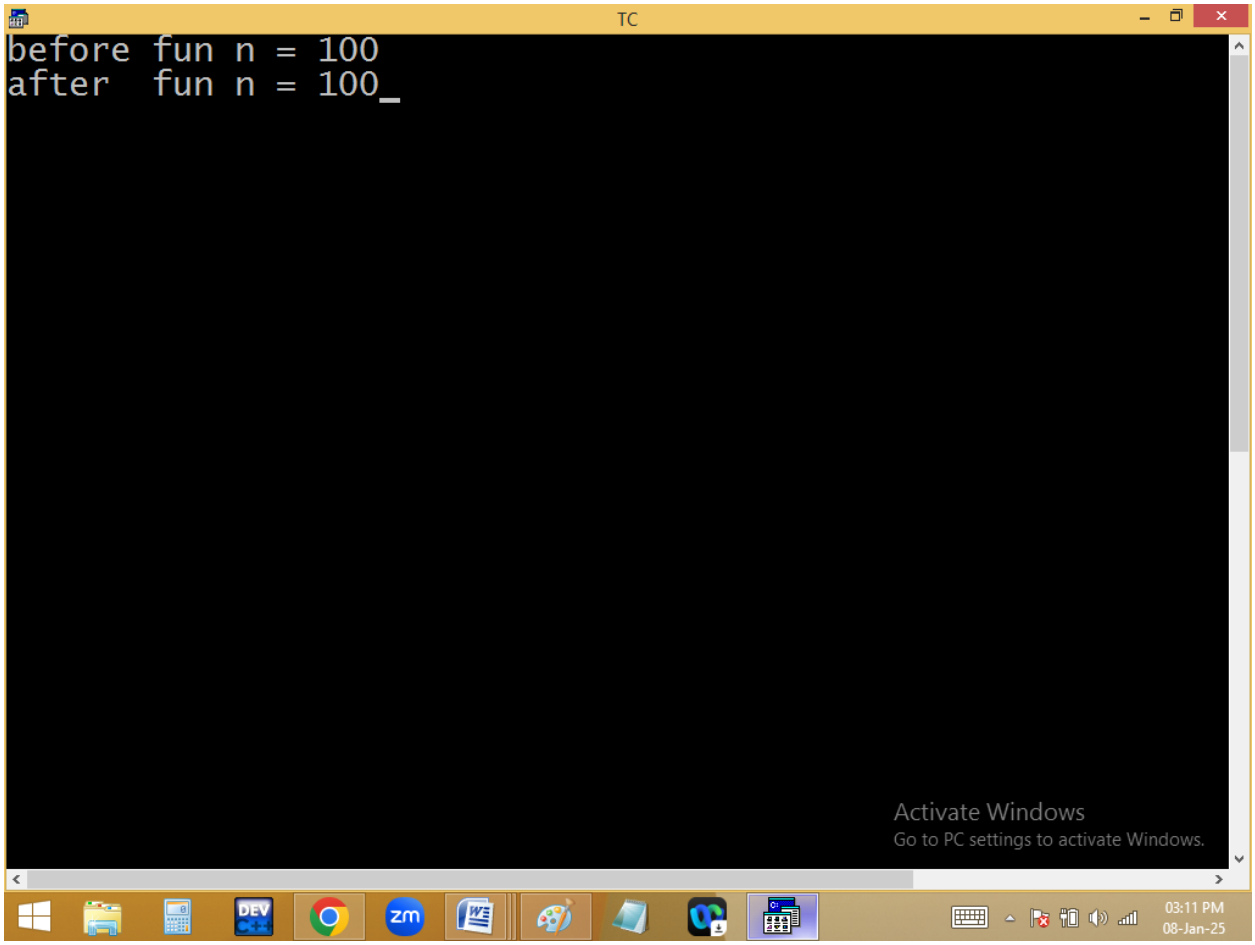
x value is 65500

* means value at 65500 = 200


TC
File Edit Run Compile Project Options Debug
Line 1 Col 19 Insert Indent Tab Fill Unindent * E
/* call by addr */
#include<stdio.h>
#include<conio.h>
void show(int * n); /* fun dec */
void show(int * n) /* parameter - local var - fun def */
{
*n=200;
} /* n deleted */
void main()
{
int n=100; /* local var */
clrscr();
printf("before fun n = %d\n",n);
show(&n); /* fun calling */
printf("after fun n = %d",n);
getch();
}
Activate Windows
Go to PC settings to activate Windows.
03:11 PM
08-Jan-25


TC
before fun n = 100
after fun n = 200
Activate Windows
Go to PC settings to activate Windows.
03:11 PM
08-Jan-25


TC
File Edit Run Compile Project Options Debug
Line 1 Col 17 Insert Indent Tab Fill Unindent * E
/* call by value */
#include<stdio.h>
#include<conio.h>
void show(int n); /* fun dec */
void show(int n) /* parameter - local var - fun def */
{
n=200;
} /* n deleted */
void main()
{
int n=100; /* local var */
clrscr();
printf("before fun n = %d\n",n);
show(n); /* fun calling */
printf("after fun n = %d",n);
getch();
}
Activate Windows
Go to PC settings to activate Windows.
03:11 PM
08-Jan-25



PASSING STRING / ARRAY TO FUNCTION

String/array is implicit pointer i.e. string / array variable stores base address. Due to this when string/array is passed to a function, implicitly base address is passed and formal parameter becomes pointer and it receives this address. Hence any change occurred in formal parameter, effects on actual parameter value also.

We can declare string / array formal parameter in 3 ways.

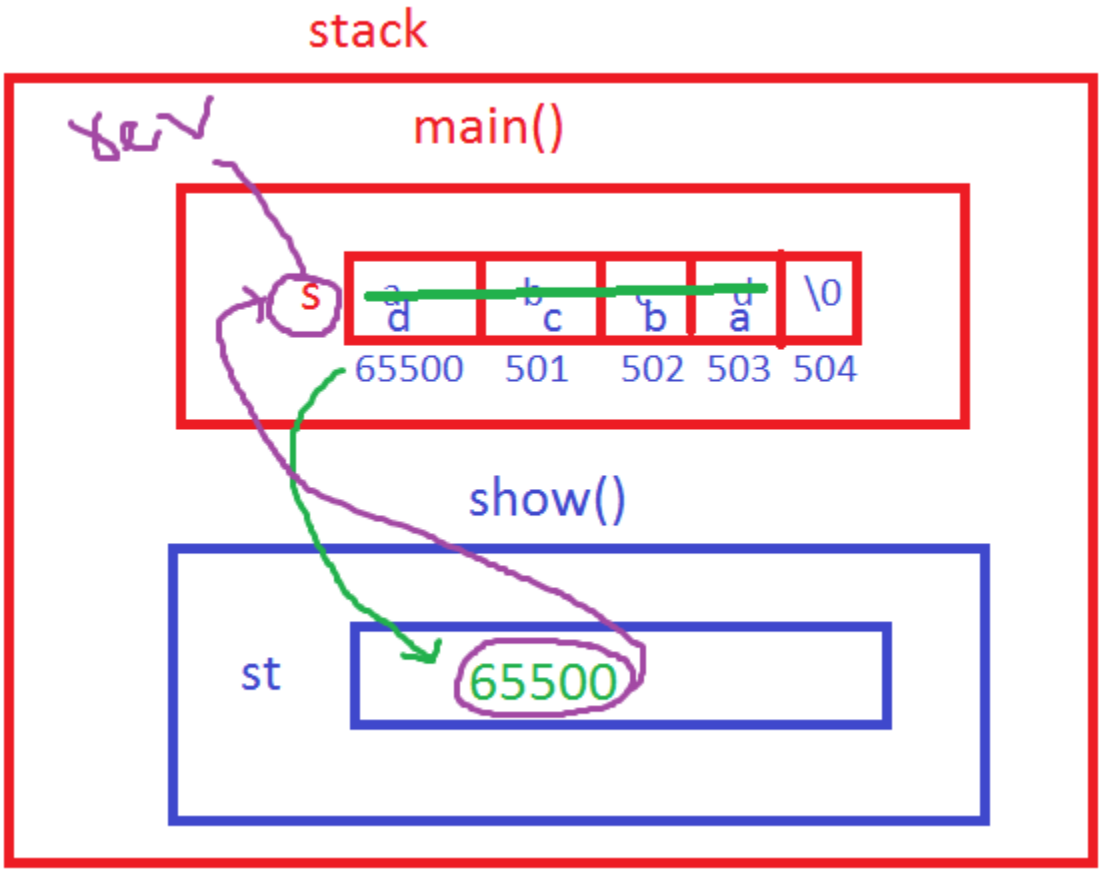
1. With size eg: char st[50] / int a[3]
2. Without size eg: char st[ ] / int a[ ]
3. As a pointer eg: char * st / int *a

We can pass string / array actual parameter with or without address.

Eg:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void  reverse( char st[10] )    or  st[ ]  or  *st
{
strrev(st);
}
void  main()
{
char  s[10]="abcd”;
clrscr();
reverse(s);  or  reverse(&s);
printf("String = %s", s);
getch();
}
```

O/P: String = dcba



Passing array to function:

#include<stdio.h>

#include<conio.h>

void show(int a[3]) or a[ ] or *a

```
{
a[0]=100; a[1]=200; a[2]=300;
}
void  main()
{
int  a[3]={10,20,30};
clrscr();
show(a);   or  show(&a);
printf("Array elements %d %d %d",a[0],a[1],a[2]);
getch();
}
```

O/P: Array elements 100   200   300

Passing two – dimensional array to function.

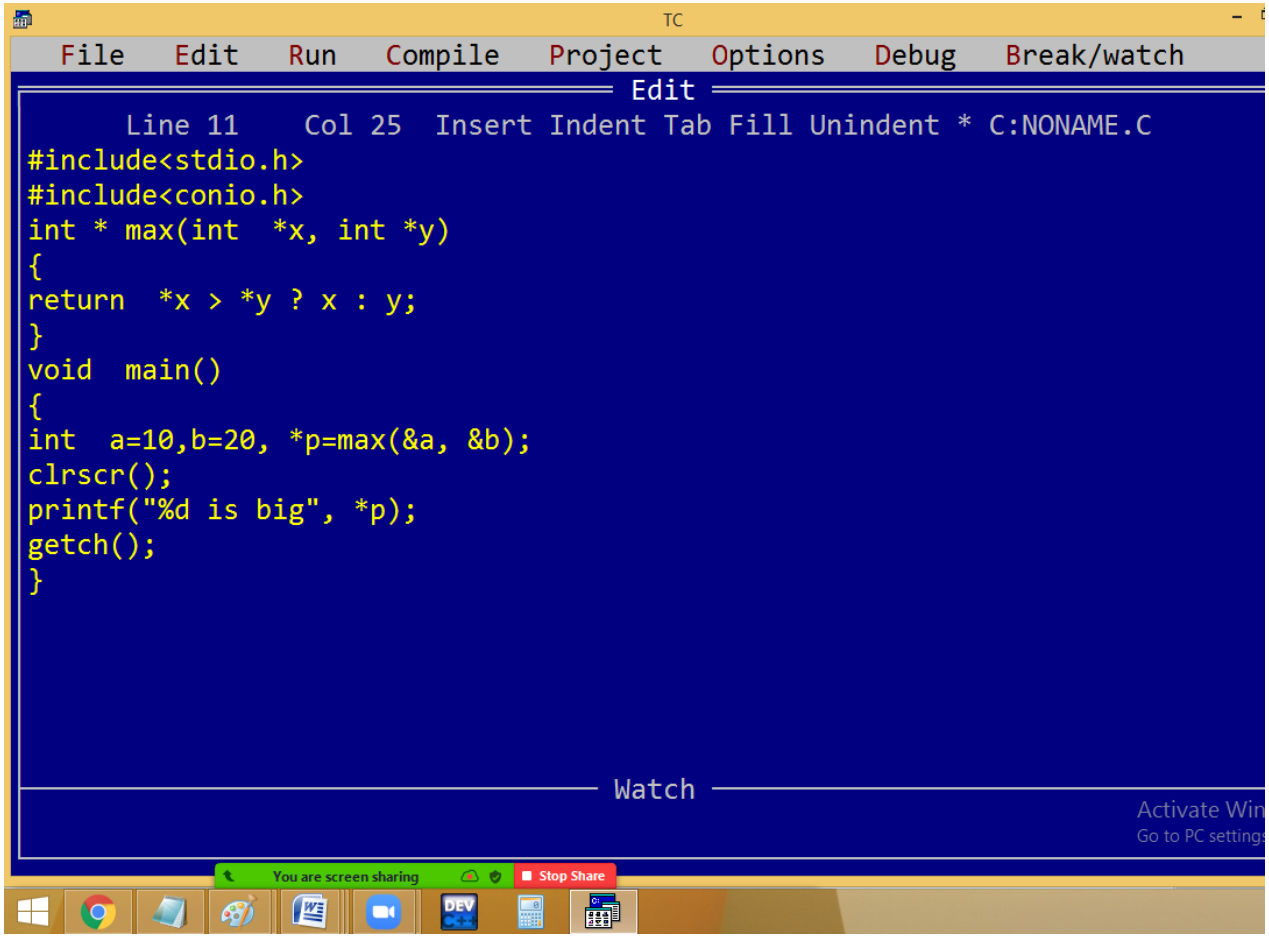
```
#include<stdio.h>
#include<conio.h>
void  show( int (*a)[3] ) or a[2][3] or a[ ][3]
{
a[0][0]=10;  a[1][2]=60;
}
void  main()
{
int a[2][3]={1,2,3,4,5,6};
show(a); /* fun calling */
printf("a[0][0]=%d, a[1][2]=%d",a[0][0],a[1][2]);
getch();
}
```

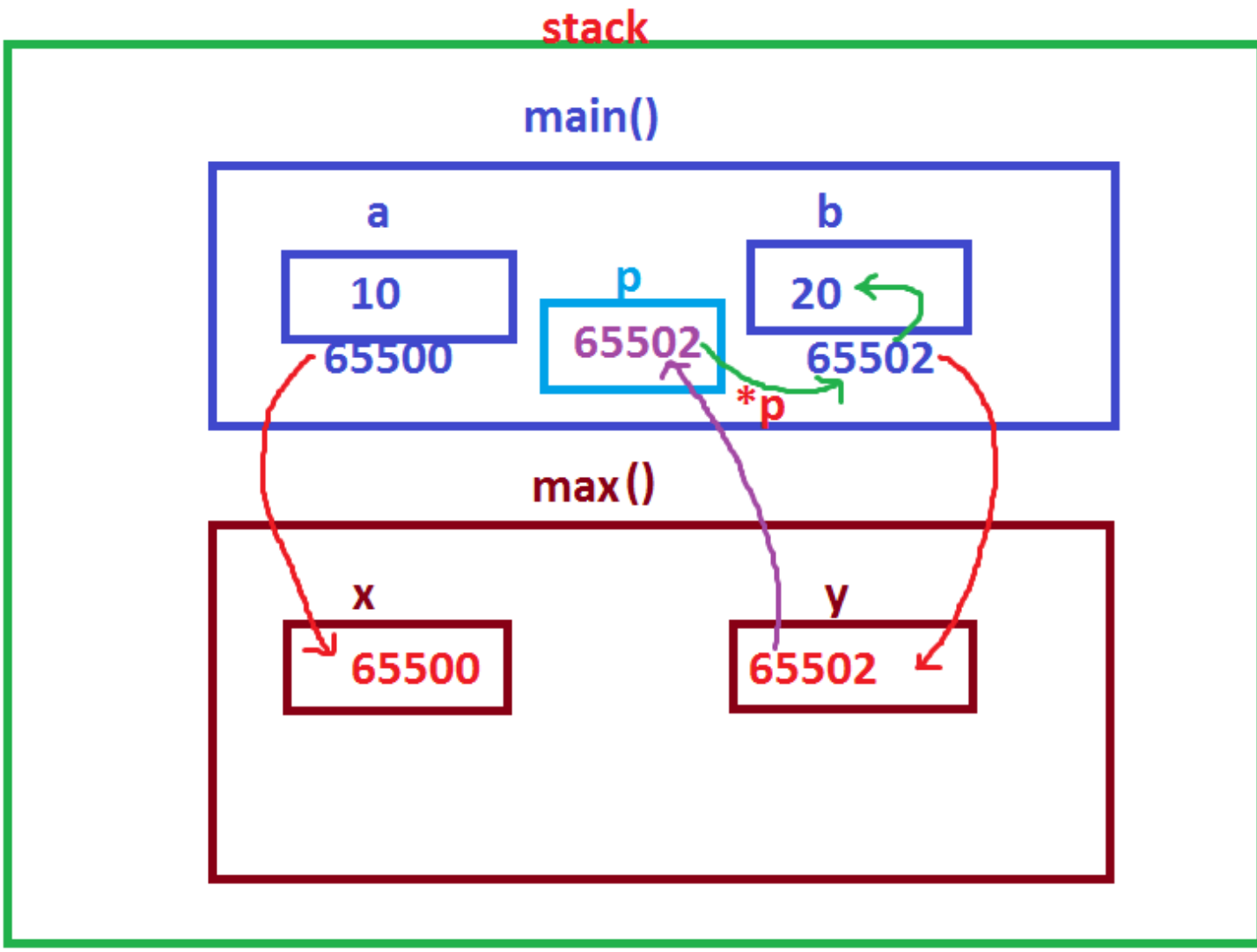
Output: a[0][0]=10, a[1][2]=60;

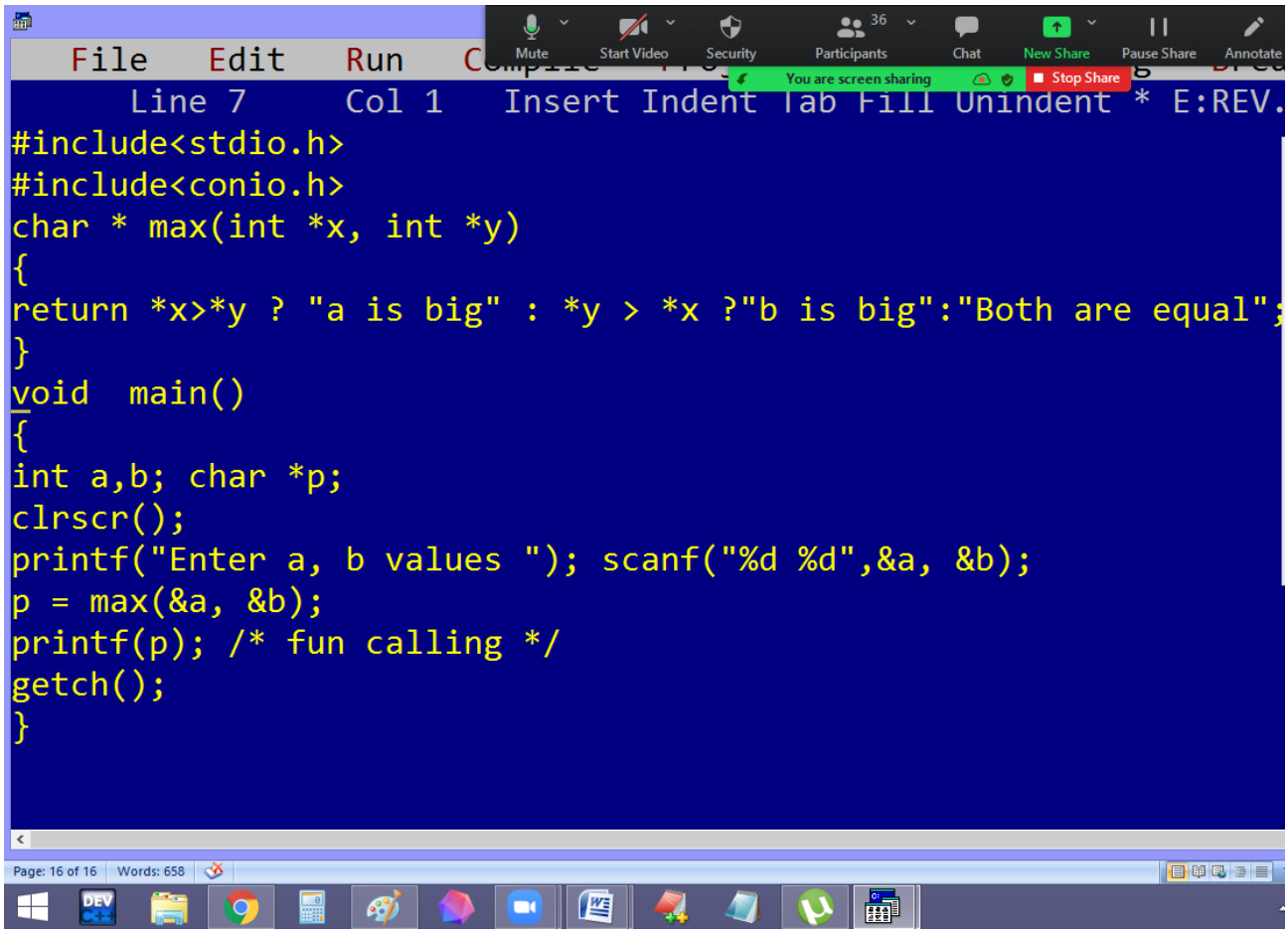
```
#include<stdio.h>
#include<conio.h>
void  show(int (*a)[3]) /* or a[2][3] or a[][3]*/
```

```
{
int  r,c;
printf("Elements are\n");
for(r=0;r<2;r++)
{
for(c=0;c<3;c++)
{
printf("%4d",*(*(a+r)+c)); /*or a[r][c]*/
}
printf("\n");
}
}
void  main()
{
int  a[2][3]={1,2,3,4,5,6};
clrscr();
show(a); /* fun calling */
getch();
}
```

Function returning address [pointer]




stack
main()
a
10
65500
p
65502
b
20
65502
*p
max()
x
65500
y
65502


Mute
Start Video
Security
36
Participants
Chat
New Share
Pause Share
Annotate
You are screen sharing
Stop Share
File Edit Run
Line 7 Col 1 Insert Indent Tab Fill Unindent * E:REV.
#include<stdio.h>
#include<conio.h>
char * max(int *x, int *y)
{
return *x>*y ? "a is big" : *y > *x ?"b is big":"Both are equal";
}
void main()
{
int a,b; char *p;
clrscr();
printf("Enter a, b values "); scanf("%d %d",&a, &b);
p = max(&a, &b);
printf(p); /* fun calling */
getch();
}
Page: 16 of 16
Words: 658


Enter a, b values 1 1
Both are equal
Mute
Start Video
Security
36
Participants
Chat
New Share
Pause Share
You are screen sharing
Stop Share
Page: 20 of 23
Words: 1,639